

# LR-DSR: clustering windows by the law that produced them

The algorithm, its building blocks, its objective, and the data it runs on. A guide to the method; results appear only where they explain a design choice.

**In one paragraph.** You have many short pieces of data (*windows*): a day of traffic, six hours of a wind turbine, one heartbeat. Each window was produced by one of  $K$  hidden rules, closed-form laws such as  $y = x^2 + x$ . Nobody says which rule produced which window. LR-DSR recovers **both**: a label for every window and one readable formula per group. It does so by alternating two easy steps, *fit a formula per group* and *move every window to the formula that explains it best*, starting from a partition computed in *mechanism space*, where windows are compared by their law coefficients rather than by how they look.

## Contents

- 1. The problem: what is given, what is unknown
- 2. The data: what a window is, simulated and real
- 3. The objective function
- 4. The algorithm, block by block (start, fit, noise scale, score, assign)
- 5. The theory: the best possible error, and how it is used
- 6. Variants: soft EM, real time, classification, other bases, declared knowledge
- 7. Assumptions and limits
- 8. Results at a glance
- 9. Where each piece lives in the code; glossary

# 1. The problem

We observe  $W$  windows. Window  $w$  holds  $n$  input/output pairs. The model is:

$$y_{wi} = f_{z_w}(x_{wi}) + \varepsilon_{wi}, \quad z_w \in \{1, \dots, K\}, \quad i = 1, \dots, n$$

| symbol             | meaning                                                          | known?                |
|--------------------|------------------------------------------------------------------|-----------------------|
| $x_{wi}$           | input of sample $i$ in window $w$ (e.g. hour of day, wind speed) | yes                   |
| $y_{wi}$           | output (e.g. traffic count, power)                               | yes                   |
| $K$                | number of regimes (laws)                                         | given by the user     |
| $f_1 \dots f_K$    | the laws, one closed-form function per regime                    | <b>no</b> : recovered |
| $z_w$              | which law produced window $w$ (the regime label)                 | <b>no</b> : recovered |
| $\varepsilon_{wi}$ | noise of scale $\sigma$ , assumed independent sample to sample   | no                    |

The whole window shares one label: all  $n$  samples of a window come from the same law. That is what makes the problem solvable. One noisy point says little, but  $n$  points that must all fit the same formula say a lot.

## Why not simply cluster the windows?

The usual recipe computes a few statistics per window (mean, spread, trend), clusters those, then fits a model per cluster. It groups windows that **look** alike. But a window's appearance depends on the inputs it happened to see. Two windows of the same law can look nothing alike, and two windows of different laws can look the same:

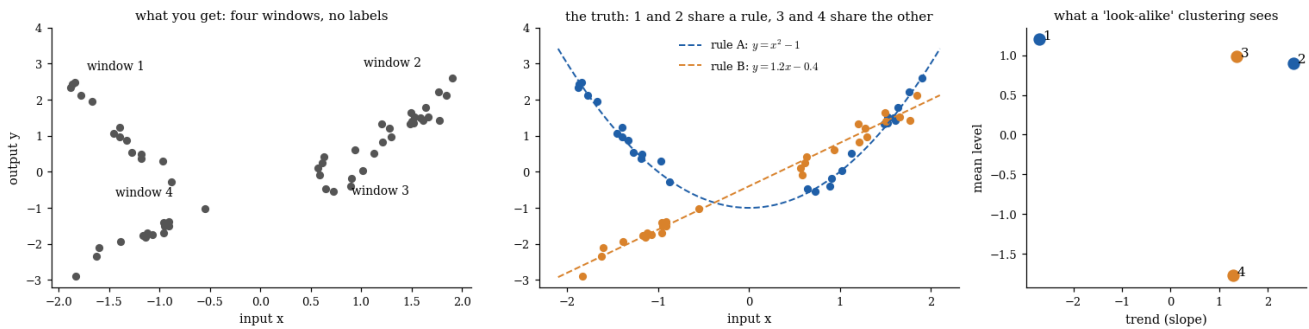


Figure 1. Four windows (left), the truth (middle): 1 and 2 share rule A, 3 and 4 share rule B. In the statistics a look-alike clustering uses (right), window 2 sits next to window 3, the wrong partner. The only thing windows 1 and 2 share is the equation that produced them.

**The principle.** The law is the cluster identity. Two windows are similar when the same equation produced them, however different they look. And the output is a readable formula per group, not just a group number.

## 2. The data

A **window** is any short chunk of data you are willing to assume follows one law: a day, a block of hours, one series. The input  $x$  need not be time; it is whatever the law is a function of.

| data                                | one window                                    | input $x$                                            | output $y$                                     | samples  | reference labels                                     |
|-------------------------------------|-----------------------------------------------|------------------------------------------------------|------------------------------------------------|----------|------------------------------------------------------|
| simulated (the zoo)                 | a batch of samples                            | drawn at random from one range shared by all regimes | a law + noise                                  | 16-48    | the truth (for scoring only)                         |
| Capital Bikeshare (DC), 2011-12     | one day (723 days)                            | hour of day, $x = 2\pi h/24$                         | $\log(1+\text{rentals})$ minus that day's mean | 24       | calendar: working day or not (a proxy)               |
| I-94 traffic (Minneapolis), 2012-18 | one day (1214 complete + 616 with gaps)       | hour of day                                          | $\log(1+\text{count})$ minus day mean          | 6-24     | calendar (a proxy)                                   |
| UCR archive                         | one series (24 datasets)                      | time index                                           | the series                                     | varies   | the archive's labels                                 |
| Kelmarsh wind farm (UK)             | one turbine, one 6-hour block (14 626 blocks) | wind speed (m/s)                                     | power as a share of rated                      | up to 36 | none exist: physical proxies (cold/warm, waked/free) |

### Simulated data: the problem zoo

Most results use simulated windows because then the truth is known and every answer can be checked. The **zoo** holds twelve two-to-four-law problems, each built to break one assumption: a law not in the library, a gap outside the library's reach, three or four regimes, two inputs, a rescaled copy, matched moments. Every regime draws its inputs from the **same** range, so *where* a window sits carries no information: the equation has to.

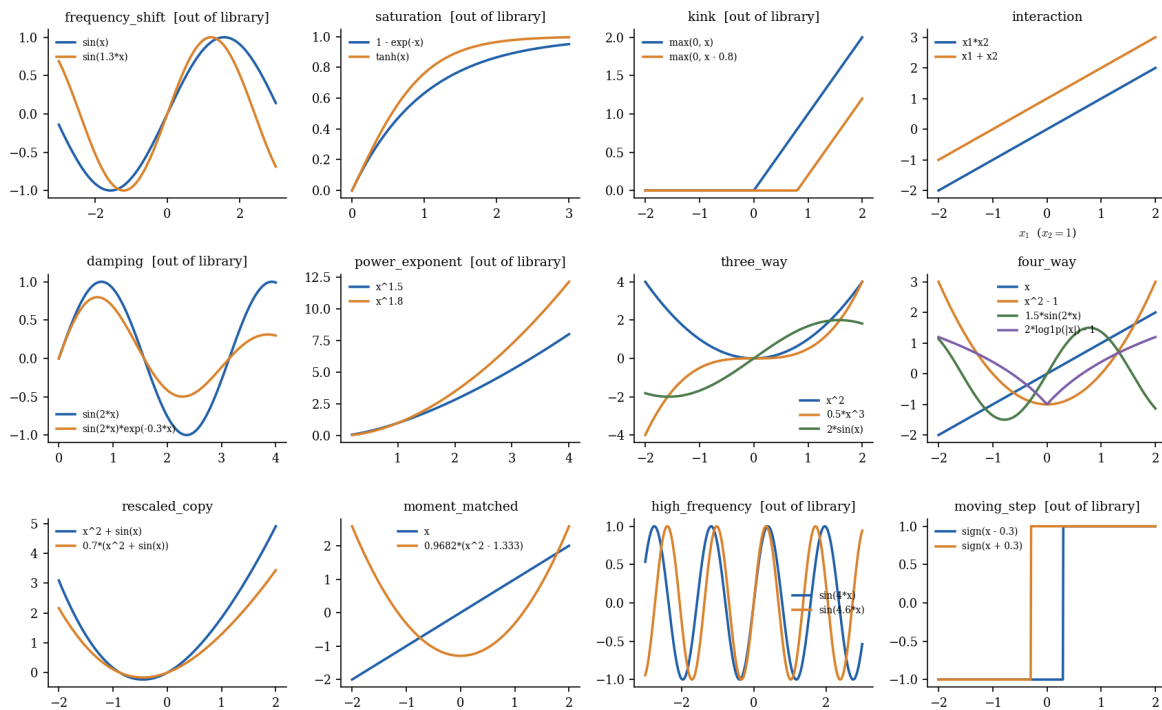


Figure 2. The twelve zoo problems. '[out of library]' marks laws the default term library cannot write exactly.

## The noise knob: separation $\rho$

Difficulty is set by one number, the separation between two laws in noise units:

$$\rho = \frac{\mathbb{E}_x[(f_1(x) - f_0(x))^2]}{\sigma^2}$$

For each problem the noise  $\sigma$  is chosen to hit  $\rho = 0.1$  (hard), 0.25 or 1.0 (easy), with the closest pair of laws at the requested  $\rho$ . Common random numbers are used: the same seed gives the same inputs and labels at every  $\rho$ , so a sweep changes the noise and nothing else.

## Real data: preparation

- **Days (bike, traffic):** one window per day,  $x = 2\pi \cdot \text{hour}/24$ ,  $y = \log(1 + \text{count})$  minus the day's mean, so the season's level is gone and a regime can only be the **shape** of the daily law. Days with missing hours are dropped, except in the experiments that are about gaps.
- **Wind blocks:** 10-minute SCADA samples grouped by turbine and 6-hour block; a sample is dropped as downtime when the wind is above cut-in (4.5 m/s) but the turbine makes under 10 kW. Every block sees different wind speeds, which is the setting the method is built for.
- **Protocol:** anything tuned is tuned on seeds  $\{3, 7, 19\}$ ; every reported number uses seeds  $\{11, 23, 42\}$ ; true labels reach only the final metric; predictions are written down before the run that tests them.

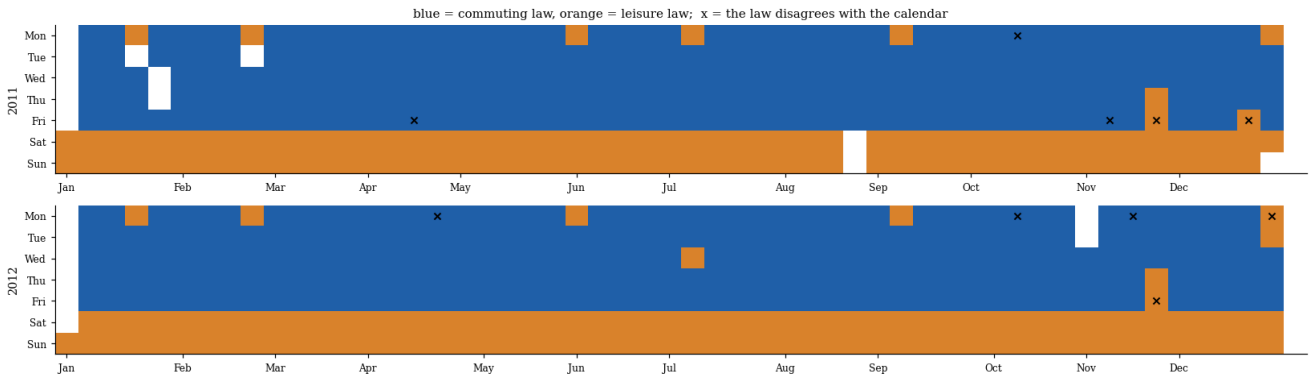


Figure 3. Real data example: two years of bike-sharing days, each coloured by the law the method found (blue: commuting shape, orange: leisure shape), with no labels given. Crosses mark days where law and calendar disagree (e.g. the Friday after Thanksgiving behaves like a day off).

### 3. The objective function

LR-DSR minimises one cost over the labels  $z$  and the laws  $f$ :

$$\min_{z, f_1, \dots, f_K} \sum_{w=1}^W J_w(z_w)$$

$$J_w(k) = \frac{1}{n} \sum_{i=1}^n \rho_{\hat{s}}\left(\frac{y_{wi} - f_k(x_{wi})}{\hat{s}}\right) + \beta C(f_k) + \alpha D_{\text{geom}}(w, k)$$

(misfit of window  $w$  under law  $k$ ) + (complexity of the law) + (optional geometry term, off by default)

| term              | what it is                                                                                                                                                                                                                                   | default                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| $\rho_{\hat{s}}$  | the per-sample <b>loss</b> . Huber: quadratic for small residuals, linear for large ones, so one outlier cannot dominate. Alternatives: squared, absolute, Cauchy, Tukey, Student-t, or a loss <b>learned</b> from the residuals each round. | Huber, $\delta = 1.345$  |
| $\hat{s}$         | <b>one shared noise scale</b> : the robust spread (MAD, in standard-deviation units) of all residuals under the current labels. Shared, so a window a law explains badly really costs more.                                                  | re-estimated every round |
| $C(f_k)$          | size of the formula: number of terms + 1                                                                                                                                                                                                     | $\beta = 0.002$          |
| $D_{\text{geom}}$ | distance of the window's summary statistics to group $k$ . With $\alpha = 0$ , assignment is by the law only.                                                                                                                                | $\alpha = 0$             |

#### The laws: what $f$ can be

Each law is a short sum of terms from a fixed **library**, with coefficients fitted by least squares. That is what makes the answer a readable formula:

$$f_k(x) = \beta_{k0} + \sum_{j \in S_k} \beta_{kj} \phi_j(x), \quad |S_k| \leq 5$$

| per input $x$                                         | for two inputs                        |
|-------------------------------------------------------|---------------------------------------|
| $x, x^2, x^3, \log(1 +  x ), \sin x, \cos x, \sin 2x$ | all pairwise products $x_a \cdot x_b$ |

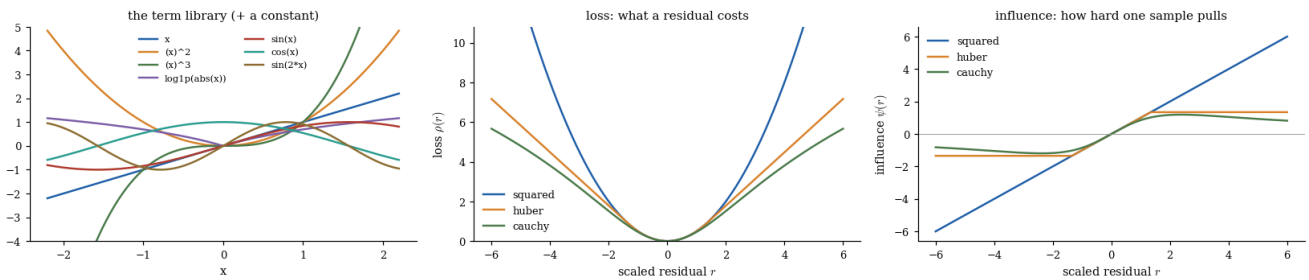


Figure 4. Left: the term library. Middle: the losses, what a residual costs. Right: their influence, how hard one sample can pull the fit; Huber caps it.

**Why this objective.** With  $\alpha = 0$  it is exactly what K-means minimises, with the centroid of a cluster replaced by a *formula*: each window pays for how badly its group's law explains it. That is why the same optimisation strategy as K-means works (Section 4).

## 4. The algorithm, block by block

The objective is hard to minimise directly: the labels are discrete, and so is the *choice* of terms in each formula. But each half is easy when the other is held fixed. The algorithm is therefore **block coordinate descent**, K-means with 'centroid' replaced by 'law': a good start, then alternate *fit laws given labels* and *assign labels given laws*.

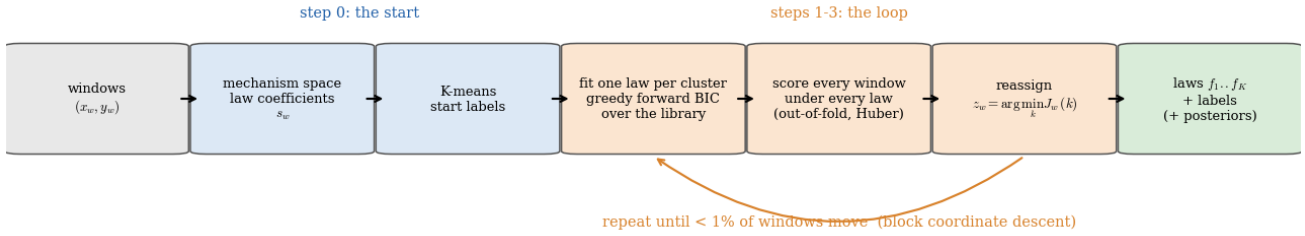


Figure 5. The pipeline. Step 0 (blue) builds the start; steps 1-3 (orange) are the loop, repeated until fewer than 1% of windows move.

### Block 0: the start, in mechanism space

The start matters, and it is not done on look-alike statistics either. Every window is mapped to its **law coefficients**, in a coordinate system where noise is equally large in every direction:

```

for every window w:  B_w = library terms evaluated at the window's inputs
remove the shared:  r_w = y_w - B_w * beta_pooled (the law every window has in common)
common footing:    G = mean_w B_w' B_w = V diag(lambda) V' (pooled Gram matrix)
                   C_w = B_w V lambda^(-1/2) (whitened terms)
the statistic:      s_w = C_w' r_w (one vector per window)
start labels:      z = K-means(s_1 ... s_W, K)
  
```

If the laws lie in the span of the library, the vectors  $s_w$  form a mixture of **round** Gaussian clouds, one per regime, with centres  $\sqrt{np}$  apart, the setting where plain K-means is the right rule. A component shared by all regimes cancels out, and a nuisance term (such as a day's level) can be profiled out first.

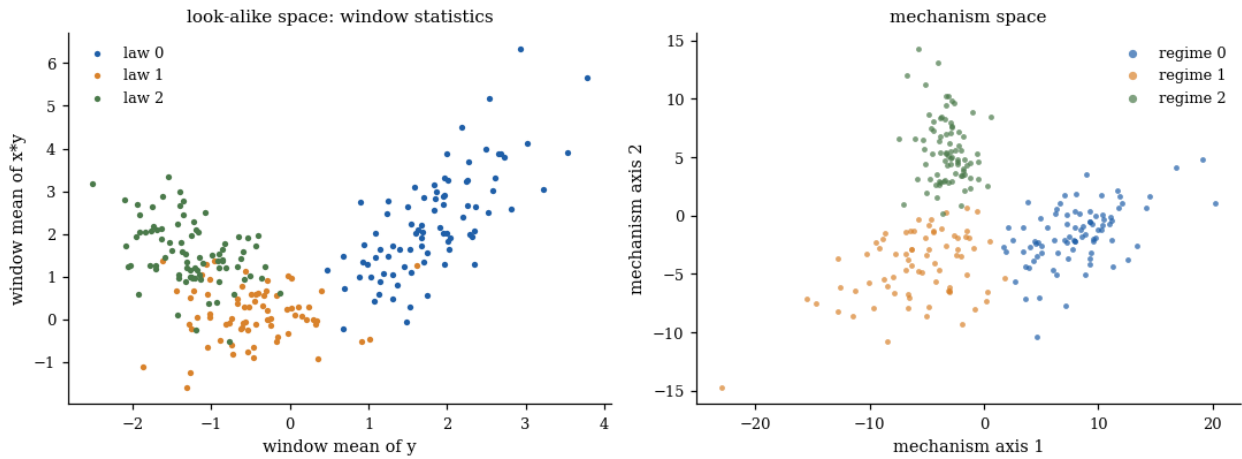


Figure 6. The same 240 simulated windows. Left: two window statistics, the colours (true regimes) are mixed. Right: mechanism space, three separate groups.

### Block 1: fit one law per group (greedy forward BIC)

With the labels fixed, pool all samples of all windows in group  $k$  and search the library:

- start from a constant;
- each round, try adding every remaining term, fit the coefficients by least squares, and keep the term that lowers BIC the most;
- stop when no term lowers BIC, or 5 terms are in.

$$\text{BIC} = N \log\left(\frac{\text{RSS}}{N}\right) + p \log N$$

$N$  is the number of pooled samples, RSS the residual sum of squares,  $p$  the number of coefficients. BIC trades fit against size, so the law stays short. Complexity is chosen **automatically**, with no hand-set term count below the cap of 5.

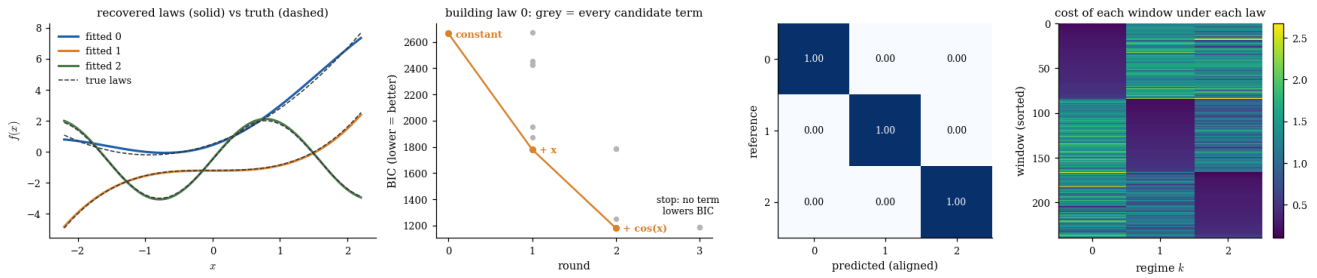


Figure 7. Left: recovered laws (solid) against the truth (dashed). Second: greedy BIC building one law; grey dots are every candidate term tried in each round. Third: confusion matrix. Right: the cost of every window under every law.

## Block 2: the noise scale

$\hat{\sigma}$  = robust spread of the residuals of every window under its current group's law (median absolute deviation  $\times 1.4826$ , i.e. in standard-deviation units). With a learned loss, the noise model (Gaussian, Laplace or Student-t) and its scale are refitted here instead.

## Block 3: score every window under every law (out-of-fold)

Fill the  $W \times K$  cost matrix  $J_w(k)$  of Section 3. One detail matters: a law fitted *on* a window flatters that window. So the entry for a window's **own** group is recomputed out-of-fold: the group is split into 3 folds, and each fold is scored by a law fitted without it (**cross-fitting**). Entries for other groups need no correction.

## Block 4: assign, repair, stop

- **assign**: every window moves to its cheapest law,  $z_w = \operatorname{argmin}_k J_w(k)$ ;
- **repair**: a group left with fewer than 4 windows takes the windows that are cheapest for it;
- **stop** when fewer than 1% of windows changed (at most 10 rounds), then refit the laws once more on the final groups.

## The whole algorithm in pseudo-code

```

INPUT  windows (X_w, y_w), w = 1..W; number of regimes K
OUTPUT labels z_w; one formula f_k per regime

0  z <- K-means( mechanism_space(X, y) )          # the start
   repair z so every group has >= 4 windows
repeat up to 10 times:
1  for k in 1..K: f_k <- greedy_BIC( pooled samples of group k )  # fit
2  s <- robust_spread( residuals under current labels )          # noise scale
3  for every window w and law k:                                # score
     J[w,k] <- mean_i Huber( (y_wi - f_k(x_wi)) / s ) + 0.002 * size(f_k)
     J[w, z_w] <- same, with f_{z_w} refitted without w's fold  # cross-fit
4  z_new <- argmin_k J[w,k]; repair tiny groups                  # assign
   if fraction(z_new != z) < 1%: stop
   z <- z_new
refit f_1..f_K on the final groups; return z, f

5  (optional) for k in 1..K: g_k <- PySR( half of group k );
   if held-out cost(g_k) < held-out cost(f_k refit on that half): f_k <- g_k
   then one more assign step

```

the loop repairs a bad start: fit laws, reassign windows, repeat

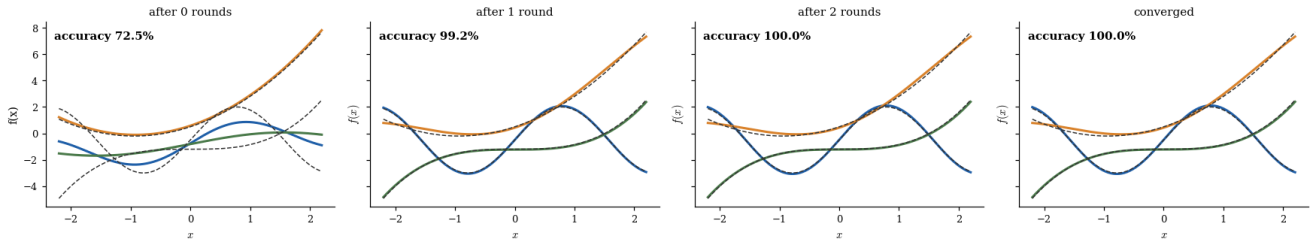


Figure 8. The loop at work, started on purpose from a bad partition (K-means on look-alike statistics): laws fitted after 0, 1, 2 rounds and at convergence, with the accuracy at each point.

**How exact is the descent?** Step 4 lowers the objective exactly, given the laws. Step 1 lowers it only approximately: it selects terms by BIC on squared error, not by the Huber cost, and step 3 deliberately departs from the in-sample cost to avoid self-flattery. In practice the loop converges in 2-5 rounds.

**What each block contributes (measured).** The **start** does most of the work: K-means in mechanism space alone is within 0.002 of the best possible error in most settings. The **loop** is a stabiliser: it repairs a bad start, barely changes a good one, and is what produces the readable formulas. The **library** is the limit: a law it cannot write comes back only as a look-alike.

## 5. The theory: the best possible error

Even an **oracle** that knows both true laws misassigns some windows, because of noise. It labels a window by the smaller residual, and its error is an exact formula:

$$P_{\text{err}} = Q\left(\frac{\sqrt{n\rho}}{2}\right), \quad Q(t) = P(Z > t), \quad Z \sim N(0, 1)$$

It depends on the window length  $n$  and the separation  $\rho$  only through their product. This ceiling is the yardstick for every result: 'good' means close to what is possible at all. It is extended in the project to what a robust loss costs (its efficiency  $\eta$  multiplies  $np$ ), to the delay of detecting a switch in real time ( $\approx 2h/\rho$  samples for threshold  $h$ ), to classification learning curves, and to windows with missing samples and correlated noise.

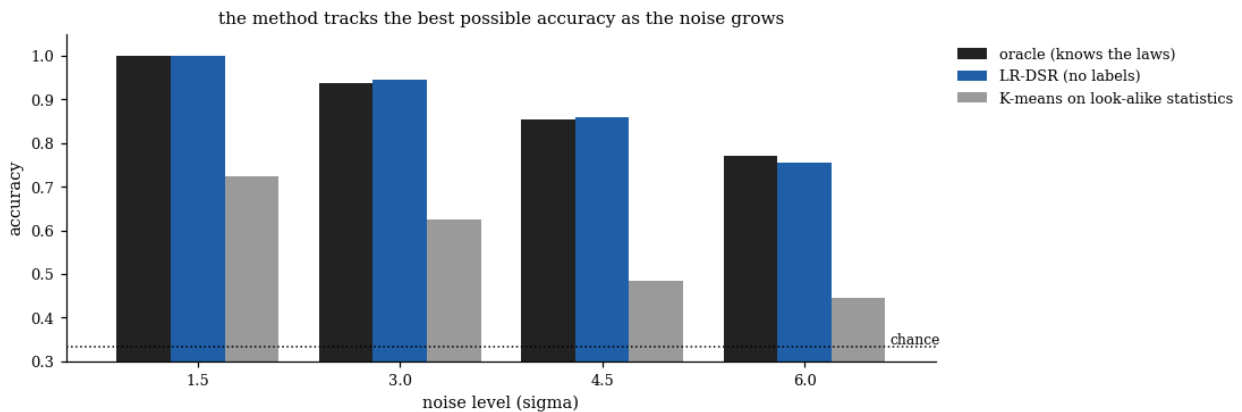


Figure 9. As noise grows, the method (no labels, laws unknown) tracks the oracle's accuracy; clustering look-alike statistics does not.

## 6. Variants: the same idea, different machinery

| variant                       | what changes                                                                                                                                                                                                                                            | use when                        |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| <b>Soft EM</b><br>(SoftLRDSR) | Each window gets a <i>probability</i> per regime instead of one label. E-step: posteriors from each law's likelihood; M-step: weighted least squares over a fixed set of terms. Gaussian or Student-t noise. A true EM: the likelihood never decreases. | you want uncertainty per window |

| variant                                     | what changes                                                                                                                                                                                                                                                                                                                                                                                                                | use when                           |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| <b>Real time</b><br>(OnlineLRDSR)           | Windows arrive one at a time. Each regime updates its coefficients in closed form (recursive least squares) in microseconds; a forgetting factor tracks drift. A window no law explains is buffered; a consistent buffer becomes a <b>new regime</b> .                                                                                                                                                                      | streams                            |
| <b>Switch detection</b><br>(CusumSegmenter) | Sample by sample, CUSUM on the log-likelihood ratio between laws; detection delay predicted by the theory.                                                                                                                                                                                                                                                                                                                  | one long stream that switches      |
| <b>Classification</b><br>(LawClassifier)    | Labels are known: fit $L$ laws per class with the same loop inside each class; a new window goes to the class whose laws explain it best.                                                                                                                                                                                                                                                                                   | labelled training windows          |
| <b>Other bases</b><br>(kernel.py)           | Fourier, cosine, Legendre, Nyström RBF, random Fourier replace the library in blocks 0 and 1. More flexible, less readable.                                                                                                                                                                                                                                                                                                 | laws far from the library          |
| <b>Declared knowledge</b>                   | Per regime: a known law; a known part + discovered rest; a known form with fitted constants; a mix of two other regimes' laws.                                                                                                                                                                                                                                                                                              | physics is partly known            |
| <b>Refine</b> (optional block 5, refine.py) | After the loop, PySR searches each final group's law once, far more deeply than block 1 (formulas built from operators, constants fitted inside). The found law replaces the loop's only if it lowers the group's cost on the <b>same</b> objective, judged on held-out windows (both laws fitted on half the group, compared on the other half, as in cross-fitting); then one more assign step. About a minute per group. | laws outside every library         |
| <b>Fitted frequency</b><br>(trial)          | Two extra term types $\sin(a \cdot x)$ , $\cos(a \cdot x)$ with the frequency $a$ fitted (grid search + refinement); BIC charges one parameter for $a$ . Only block 1's vocabulary changes; objective and loop are unchanged.                                                                                                                                                                                               | oscillations faster than $\sin 2x$ |

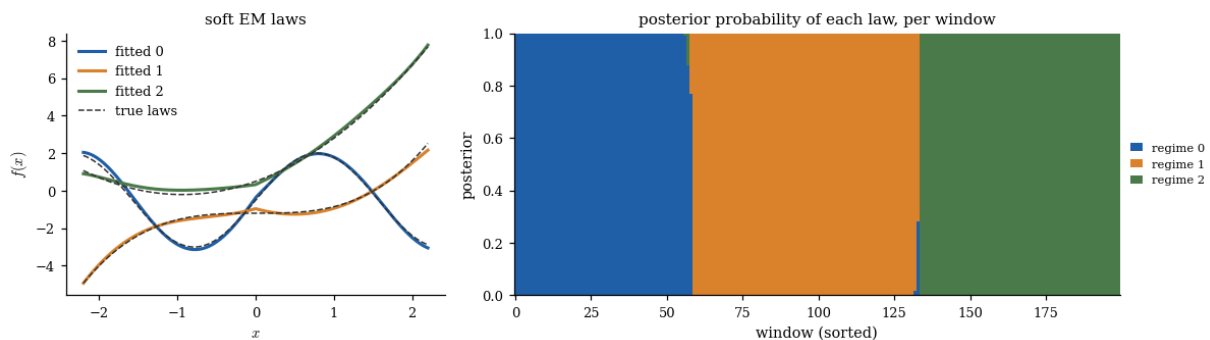


Figure 10. Soft EM on the worked example: recovered laws and the posterior probability of each law for every window (sorted).

## 7. Assumptions and limits

| assumption                                | what happens when it fails                                                                                                                                                                                                                                   |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The number of regimes $K$ is known.       | Must be given. Choosing it by BIC was tried and fails on real days (it keeps asking for more).                                                                                                                                                               |
| Each law is a short sum of library terms. | A law outside the library comes back as a look-alike built from library terms; assignment degrades in proportion to how much of the <i>difference</i> between laws the library cannot see. Remedies: kernel bases, the fitted-frequency term.                |
| Noise is independent sample to sample.    | Correlated (wandering) noise means fewer effective samples. The method still works at the correlation real days show ( $\text{lag-1} \approx 0.35\text{-}0.43$ ), but falls behind the oracle as correlation grows; confidence and BIC become overconfident. |
| One law per window.                       | A switch inside a window is not modelled (CUSUM handles switches in a stream).                                                                                                                                                                               |
| Laws are static maps $y = f(x)$ .         | No dynamics (differential equations, autoregression).                                                                                                                                                                                                        |

| assumption                              | what happens when it fails                                                                              |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------|
| The inputs carry no regime information. | Holds by design in the simulations; if they do, the optional geometry term ( $\alpha > 0$ ) can use it. |

same law, same noise level, same random numbers -- only the correlation differs

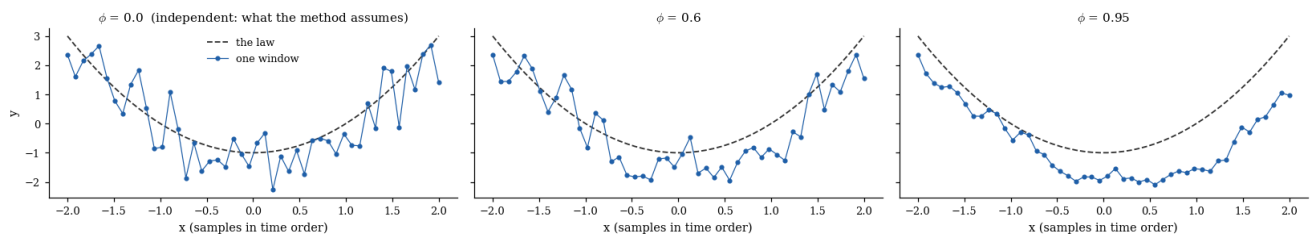


Figure 11. What 'independent noise' means. Same law, same noise level: independent (left, the assumption), then increasingly wandering noise ( $\phi$  = lag-1 correlation).

## What is not in the package

- open-ended formula discovery inside the loop (too slow; PySR runs only in the optional block 5, and no reported result uses it);
- automatic choice of  $K$ ;
- a correction for time-correlated noise (it is measured, not corrected);
- comparisons with mixtures of spline regressions, Gaussian-process mixtures, DTW or k-Shape, and hidden Markov models.

## 8. Results at a glance

Each panel is one question the project asked; frame colour is the verdict (green: holds, amber: partly). Lower is better unless the panel says otherwise.

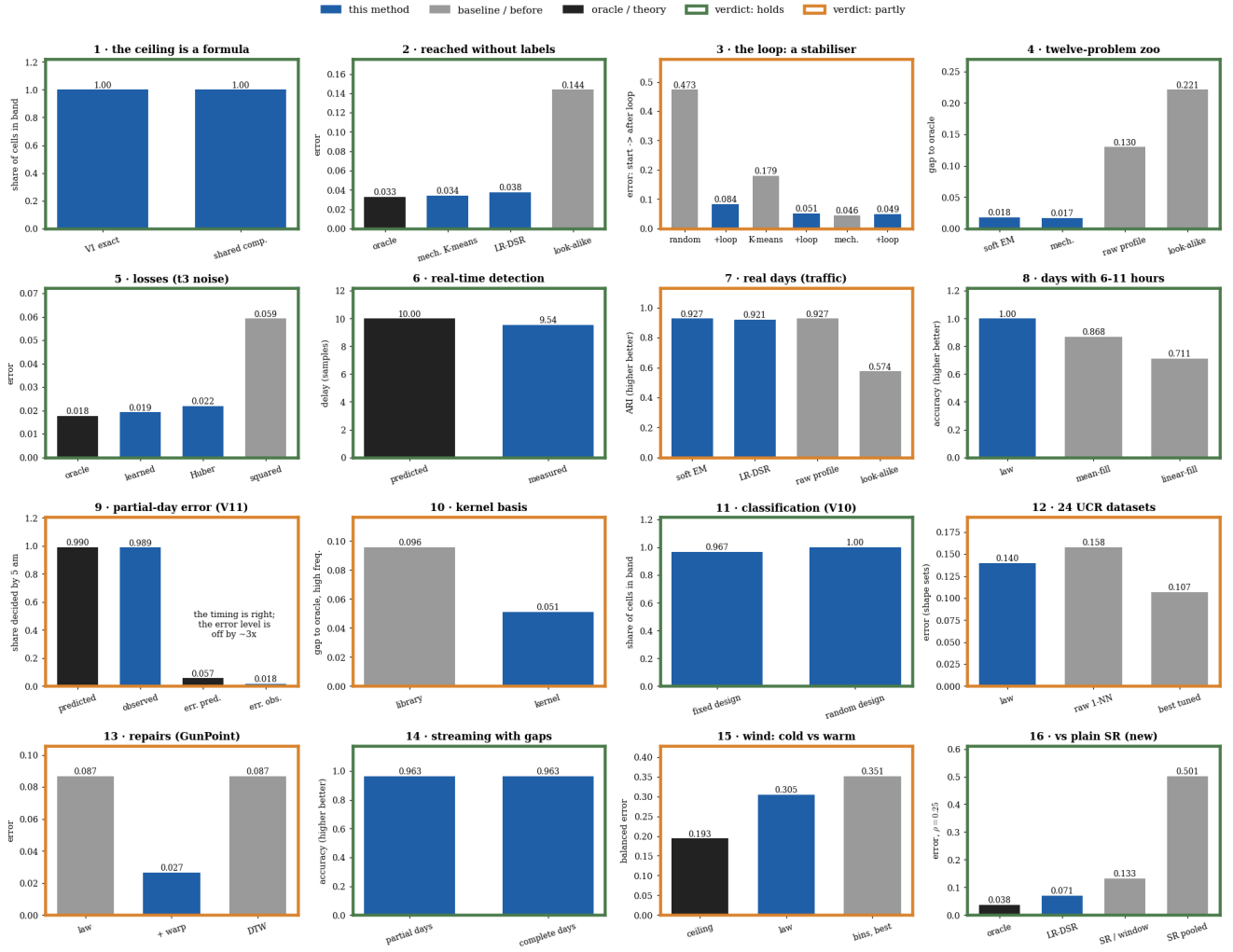


Figure 12. Every task in one picture: this method (blue) against baselines (grey) and the oracle or theory (black).

- **Reaches the ceiling without labels** on laws the library can express.
- **Wins where the inputs differ between windows:** days with long sensor gaps (6-11 of 24 hours: law 1.000 vs 0.87 / 0.71 for filling in the missing hours), wind turbines at different wind speeds, noisy oscillations.
- **Ties or loses** when every window has the same complete inputs: a K-means on the raw profile ties it on whole days; tuned standard classifiers beat it on the UCR archive.
- **Beats plain symbolic regression:** one law per window is too noisy, one law for all windows is wrong; pooling the windows of a regime is what makes the laws accurate.

## 9. Where each piece lives; glossary

| building block                                                        | code                                      |
|-----------------------------------------------------------------------|-------------------------------------------|
| objective, loop, cross-fitting, repair                                | lrdsr/core/model.py (GroupedDCSR)         |
| mechanism space (the start)                                           | lrdsr/core/mechanism_space.py             |
| library and greedy BIC search; fitted-frequency term; knowledge modes | lrdsr/core/backends.py                    |
| losses, noise scale, joint cost                                       | lrdsr/core/losses.py                      |
| soft EM                                                               | lrdsr/core/soft.py                        |
| real time, CUSUM                                                      | lrdsr/core/online.py                      |
| classification                                                        | lrdsr/core/classify.py                    |
| kernel and function bases                                             | lrdsr/core/kernel.py                      |
| theory: ceiling, losses, delay, learning curve, partial windows       | lrdsr/theory/                             |
| experiments (one block per question)                                  | experiments/ (python -m experiments list) |
| the end-to-end notebook                                               | notebooks/00_summary.ipynb                |

| term                     | meaning                                                                              |
|--------------------------|--------------------------------------------------------------------------------------|
| window                   | a short chunk of data assumed to follow one law                                      |
| regime / law             | one of the K hidden rules; recovered as a formula                                    |
| library                  | the fixed list of terms formulas are built from                                      |
| mechanism space          | windows represented by their whitened law coefficients                               |
| separation $\rho$        | how far apart two laws are, in noise units; sets the difficulty                      |
| oracle                   | a rule that knows the true laws: the best possible accuracy                          |
| BIC                      | Bayesian information criterion: fit versus size, used to choose each formula's terms |
| cross-fitting            | scoring a window with a law fitted without it                                        |
| Huber loss               | squared for small residuals, linear for large: robust to outliers                    |
| block coordinate descent | optimise one group of unknowns at a time, holding the others fixed                   |